TP D: Desarrollo de aplicaciones distribuidas con BSD sockets

Fecha y hora límite de entrega: 20/11 23:59:59
Carácter: grupal (grupos de 3).

Objetivo **0**
Conformación de grupos **0**
Desafíos **1**
Primer desafío: autenticarse para solicitar puerto TCP 1
Pautas: 1
Formato y procedimiento de interacción 1
Respuesta exitosa 2
Respuesta fallida 2
Diagrama de secuencia 3
Segundo desafío: conectarse por TCP y delimitar PDUs 4
Diagrama de secuencia 6
Pruebas a superar 6
Preguntas a contestar **7**
Entregables requeridos **7**

Objetivo

Que los alumnos se familiaricen con la biblioteca BSD Sockets y se interioricen con las particularidades de TCP y UDP. Que logren intercomunicar sistemas implementando una aplicación distribuida sencilla.

El trabajo práctico consta de 2 desafíos:

- Un primer desafío parte que está enfocada en practicar la programación de un cliente UDP y el uso de una API HTTP simple con un socket TCP. Checkpoint #1 3/11/2026
- En el segundo desafío se hará énfasis enfrentar la problemática del Framing en TCP y validación de Protocol Data Units. Entrega Final 20/11/2026

Conformación de grupos

Cada grupo deberá enviar un **único** correo electrónico con el asunto 'grupo TP D'. El remitente debe incluir en copia (CC) las direcciones de correo de todos los demás integrantes del grupo. El mail debe ser enviado a nmatsunaga@udesa.edu.ar y jbozzuto@udesa.edu.ar según corresponda

La cátedra proveerá por mail credenciales de servidor para el grupo y credenciales para cada grupo

Desafíos

La cátedra implementará un servidor al que deberán conectarse y responder los desafíos. El servidor estará esperando mensajes tanto en UDP, como conexiones en TCP. Los datos del servidor serán comunicados a través del campus. A su vez, habrá una web para ver si los desafíos fueron resueltos exitosamente.

Primer desafío: autenticarse para solicitar puerto TCP

Pautas:

- El servidor de la cátedra escucha en UDP en el puerto 20262 (*cliente UDP*)
- El cliente debe conocer su IP pública haciendo una consulta a la API básica http://ifconfig.me o similar http://api.ipify.org) ya que este será un dato a validar por el server. Está **prohibido usar libcurl** o bibliotecas que hablen HTTP. (*cliente TCP*)
- El cliente contará con una credencial (usuario y secreto) por grupo que será usada en la autenticación que serán provistos por la cátedra
- El cliente usará su secreto compartido en el contexto de un digesto MD5 (está prohibido usar bibliotecas externas o llamadas exteriores)
- Se deberá tener la captura de wireshark del desafío exitoso (es un requisito excluyente para la aprobación)
- Si es necesario use wireshark para hacer troubleshooting de tráfico filtrado. Esto último es una posibilidad ya que se utilizan ports no estándar y en ámbitos académicos *puede* haber filtros de tráfico.

Formato y procedimiento de interacción

A continuación se especifica el formato y el procedimiento de interacción:

La petición del cliente tiene el siguiente formato:
`<USERNAME><PIPE><IPv4ADDRESS><PIPE><SIGNATURE>`

- donde `<USERNAME>` es el nombre del grupo asignado por la cátedra que utiliza solamente caracteres ASCII de los siguientes rangos `0x30-0x39, 0x41-0x5A, 0x61-0x7A` [0-9A-Za-z]
- `<PIPE>` que es el ASCII char `0x7C`
- `<IPv4ADDRESS>` es una dirección IP expresada en Decimal Dot Notation en ASCII. Por ejemplo, 8.8.8.8 es serían los bytes: `0x38 0x2E 0x38 0x2E 0x38 0x2E 0x38`
- `<SIGNATURE>` es el digesto MD5 en caracteres hexadecimales en minuscula del ASCII (`0x30-0x39, 0x61-0x66`) del siguiente string:
  `<USERNAME><PIPE><IPv4ADDRESS><PIPE><SECRET>`
  Por ejemplo: `grupo01|180.11.1.3|secreto01` se generaría la Protocol Data Unit `grupo01|180.11.1.3|f21d11f08cf40377860fd75d8bfd73cb`

El servidor responderá con el formato:
`<CODE><PIPE><ACTION><PIPE><TEXT><PIPE><SIGNATURE>`

Respuesta exitosa

Si está todo OK <CODE> vale el string ASCII “200”, <ACTION> vale “Port” y en <TEXT> va el port number que el servidor dinámicamente abrió para esta petición. Este puerto es dinámico y se quedará escuchando esperando una conexión solamente durante 10 segundos. Si no hay una petición de conexión en dicho lapso se cerrará. A su vez, este mensaje estará debidamente protegido calculando el `<SIGNATURE>` como el digesto MD5 del siguiente string: `<CODE><PIPE><ACTION><PIPE><TEXT><PIPE><SECRET>`
donde `<SECRET>` es el secreto correspondiente a <USERNAME>. Por ejemplo,
`200|Port|17189|6d720e86a353852c268b2071dffaf069`
es decir `md5(“200|Port|17189|secreto01”) = 6d720e86a353852c268b2071dffaf069`

Si el mensaje no tiene un <SIGNATURE> válido debe ser descartado silenciosamente. El cliente debe esperar la respuesta durante 10 segundos.

Respuesta fallida

Ante una falla no se puede conocer el secreto del usuario en cuestión por lo cual el campo <SIGNATURE> es irrelevante será llenado por 32 “0”.

Si hubo un error de formato <CODE> vale el string ASCII “400”, <ACTION> vale “Message” y en <TEXT> aparece la descripción del error. Por ejemplo:

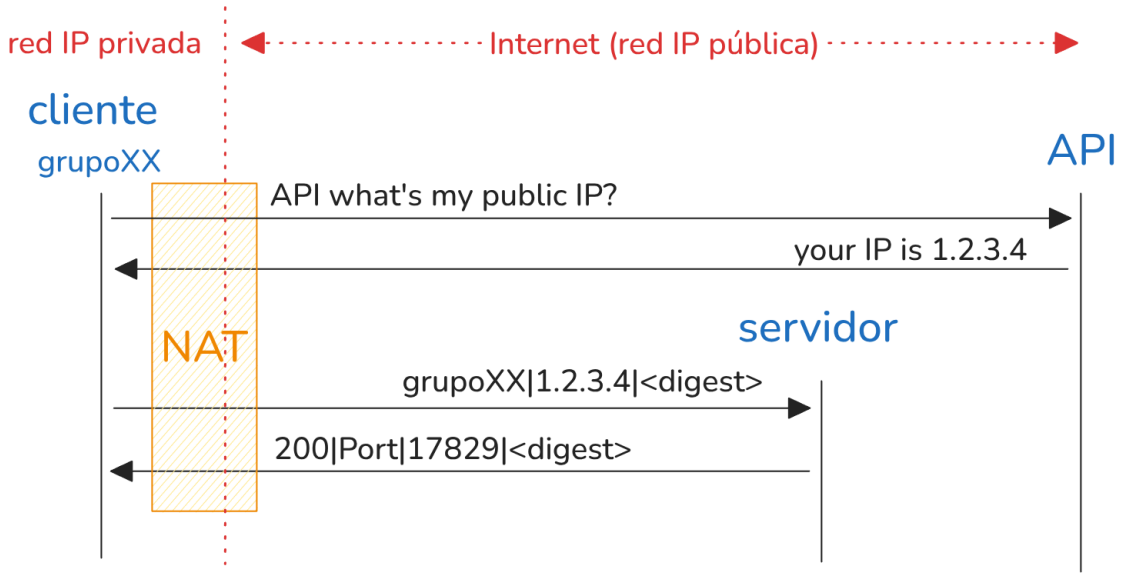
```
400|Message|Bad format|00000000000000000000000000000000
```

Si hubo un error de autenticación <CODE> vale el string ASCII “401”, <ACTION> vale “Message” y en <TEXT> aparece la descripción del error

```
401|Message|Unauthorized|00000000000000000000000000000000
```

CHECKPOINT #1 a verificar el 03/11/2026

Diagrama de secuencia



Segundo desafío: conectarse por TCP y delimitar PDUs

El cliente se conecta al Port indicado en la respuesta exitosa
El servidor enviará varias PDUs (una cantidad aleatoria) en la misma conexión TCP establecida y finalizará la conexión. A medida que el cliente logre parsear y delimitar la PDU deberá enviarlas encapsuladas en UDP y firmadas. El servidor podría enviar PDUs inválidas. Deben enviarse solamente las PDUs delimitadas que no contengan el caracter 0x7C (no haremos byte stuffing para simplificar). El formato del mensaje que cliente enviará al servidor por UDP es:
`<USERNAME><PIPE><PDU><PIPE><PARSE_CODE><PIPE><SIGNATURE>`

- donde `<USERNAME>` es el nombre del grupo asignado por la cátedra que utiliza solamente caracteres ASCII de los siguientes rangos `0x30-0x39, 0x41-0x5A, 0x61-0x7A` [0-9A-Za-z]
- `<PIPE>` que es el ASCII char `0x7C`
- `<PDU>` es la PDU parseada
- `<PARSE_CODE>` es el código de status de parseo de la PDU. Ver más abajo
- `<SIGNATURE>` es el digesto MD5 en caracteres hexadecimales en minúscula del ASCII (`0x30-0x39, 0x61-0x66`) del siguiente string:
  `<USERNAME><PIPE><PDU><PIPE><PARSE_CODE><PIPE><SECRET>`

Una PDU válida enviada en la capa de transporte tiene el siguiente formato:
`<TIMESTAMP><AMPERSAND><MESSAGE><END_DELIMITER>`

- `<TIMESTAMP>` que es el unix timestamp usando los digitos decimales de ASCII (ejemplo: 1792303544 representa Sun Oct 18 03:05:44 AM -03 2026). La longitud máxima son 11 caracteres
- `<AMPERSAND>` que es el ASCII char 0x26
- `<MESSAGE>` que son desde 1 hasta 100 caracteres ASCII en el rango 0x20- 0x7D **excluyendo** 0x26 y 0x7C
- `<END_DELIMITER>` que es la secuencia de ASCII chars `0x0D 0xA` (CR LF)
- El tamaño total máximo de una PDU incluyendo delimitadores es 122 Bytes

En función de esta definición, al intentar parsear la PDU los posibles estados son

- 200 Ok (cuando la PDU respeta los caracteres, cantidad y longitud de campos)
- 405 Invalid Timestamp (cuando se usan caracteres inválidos, fuera del rango 0x30-0x39)
- 406 Expired Timestamp (si el timestamp recibido tiene más de 300 segundos que la hora actual)
- 407 Timestamp Too Long cuando se usan más de 11 caracteres permitidos)
  408 Invalid Chars on Message (cuando se usan caracteres fuera del rango 0x20-0x7D **excluyendo** 0x26 y 0x7C)
- 409 Message Empty cuando se recibe el campo Message vacío
- 410 Message Too Long cuando se usan más de 100 caracteres permitidos)
- 400 Bad Request (cuando hay un error no tipificado en los anteriores mensajes)

Ejemplos:
Servidor envía: `1792303544&Temperatura ambiente 11 grados<CR><LF>`

Timestamp actual: 1792303547
Status: 200

Servidor envía: `12345.1&Temperatura ambiente 11 grados<CR><LF>`
Timestamp actual: 1792303547
Status: 405
Causa: Timestamp tiene caracteres fuera del rango 0x30-0x39

Servidor envía: `1762303544&Temperatura ambiente 11 grados<CR><LF>`
Timestamp actual: 1792303547
Status: 406
Causa: Timestamp tiene más de 300 segundos de antigüedad

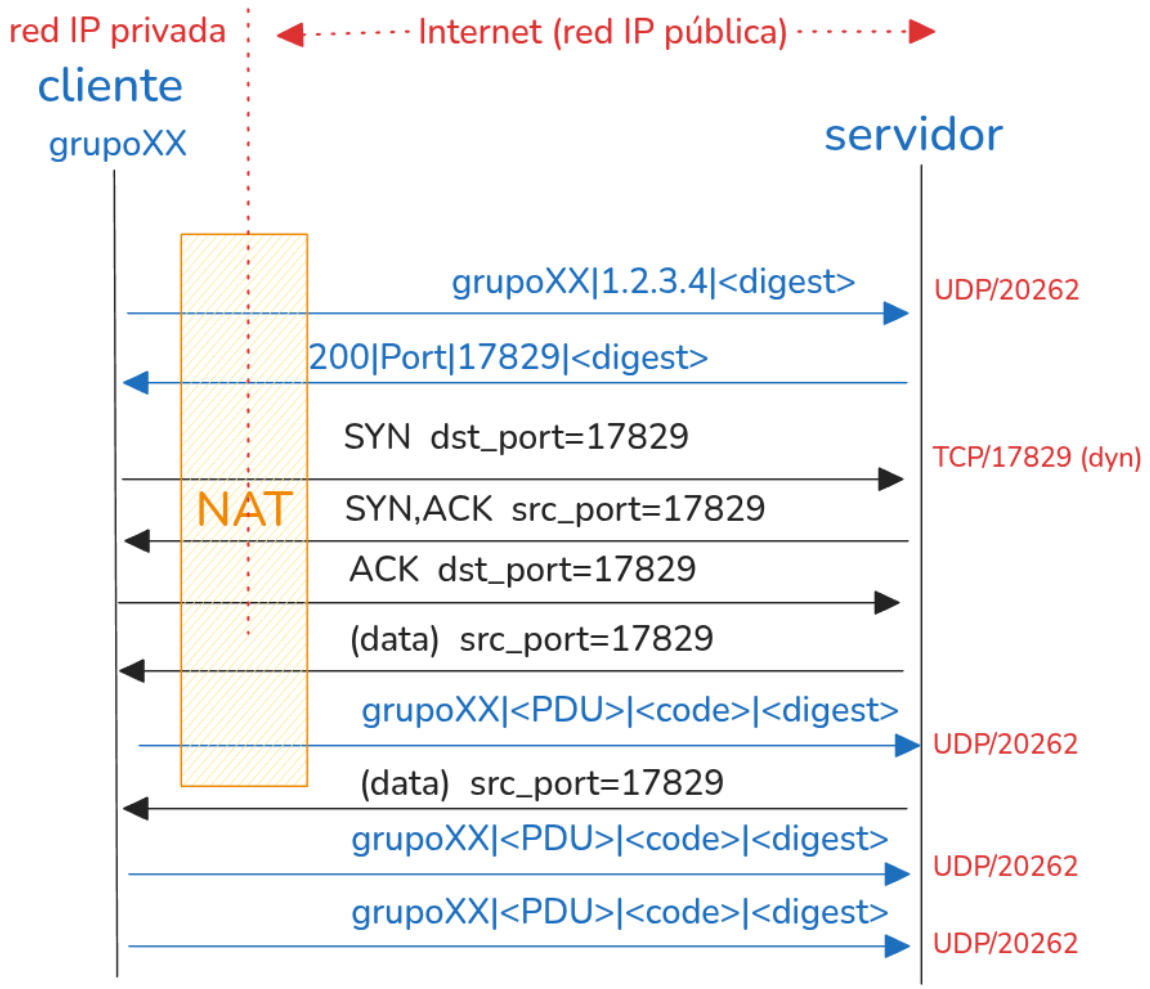
Servidor envía: `1111762303544&Temperatura ambiente 11 grados<CR><LF>`
Timestamp actual: 1792303547
Status: 407
Causa: Timestamp tiene más de 11 caracteres

Servidor envía: `1792303544&Temperatura ambiente 11 °C<CR><LF>`
Timestamp actual: 1792303547
Status: 408
Causa: caracter inválido en campo Message

Servidor envía: `1792303544&<CR><LF>`
Timestamp actual: 1792303547
Status: 409
Causa: está vacío el campo Message

Servidor envía: `1792303544^Temperatura ambiente 11 °C<CR><LF>`
Timestamp actual: 1792303547
Status: 400
Causa: No hay 2 campos

Diagrama de secuencia



Pruebas a superar

- El programa debe soportar un manejo adecuado del framing de PDU
  - el servidor podría enviar una PDU contenida en múltiples segmentos (menos de 1 PDU por segmento TCP)
  - el servidor podría enviar muĺtiples PDUs en un segmento (más de 1 PDU por segmento TCP
  - la combinación de ambos casos (una cantidad no entera de PDUs en 1 segmento)
- Pasar todos los desafíos
  - autenticación
  - conexión a un puerto establecido
  - parseo de PDUs y su correcto reenvío firmado

Preguntas a contestar

1. ¿Cómo podría mejorar el protocolo para permitir el reenvío de PDUs por UDP con el carácter prohibido usado para la delimitación 0x7C?
2. ¿Podría cambiarse el rol de quien inicia la conexión TCP para el segundo desafío? Es decir, ¿podría el servidor iniciar la conexión TCP contra sus computadoras con el programa de Sockets? Argumente su respuesta.
3. ¿Cómo se vería afectado el funcionamiento de los distintos desafíos del TP ante la pérdida de paquetes? Justifique
4. Detalle las dificultades y aprendizajes obtenidos durante el desarrollo de TP. Enumere consejos hacia futuros alumnos que encaran por primera vez la programación en BSD sockets.

Entregables requeridos

- Se deberá entregar el **código fuente** junto con sus instrucciones de uso y compilación. El código no debe contener Warnings (`-Wall`)
- Cuando se ejecuten los desafíos **se debe capturar el tráfico** con un filtro que permita reducir el tráfico no deseado. Por ejemplo: filtrar por la IP del servidor.
- Se deberá entregar un documento **PDF** con las respuestas a las preguntas y otros entregables especificados a lo largo del enunciado.

Habrá una defensa oral del trabajo práctico a todos los integrantes del grupo. Las preguntas estarán relacionadas a los entregables (códigos, capturas y documentos requeridos). Es obligatorio conocer el **funcionamiento detallado del código**, sino deberemos suponer desconocimiento del tema.